

SENTINEL

Cyber Attack Path Prediction and Risk Assessment Framework

Implementation Plan — Revision 2

Final Year Project, Dept. of ISE, DSCE · August 2026 – June 2027

Why this revision exists

Revision 1 described a sensible set of phases, but review of it surfaced four problems that would have been fatal at evaluation: the dataset was circular, the train/test split would have leaked, training and runtime were merged into a single flow, and the project scope was never actually defined.

This revision fixes those four, answers the open questions raised in team discussion, and adds three sections that were missing entirely: an evaluation plan, a stated research contribution, and a demo script.

Sections marked NEW did not exist in Revision 1. Sections marked CHANGED differ materially and should be re-read even if you read Revision 1.

Summary of changes from Revision 1

Area	Revision 1	Revision 2
Scope	Undefined — implied all attack types	Locked to one named four-stage intrusion chain
Dataset	Scripted fixed sequence	Branching scenarios + negative/benign paths
Split	Random split of event rows	Experiment-level, held-out scenario variants
Data flow	One merged flow	Three separate flows: build-time, training, runtime
Graphs	"Attack graph" used for two things	Knowledge graph / environment graph / attack state
Phase order	Attack graph before knowledge graph	Knowledge graph first
Lab	VMs, Windows endpoint, Sysmon	Containers, Linux only, single telemetry schema
Attack execution	Not specified	Caldera or Atomic Red Team, logs double as labels
Evaluation	Absent	Metrics, baselines and latency SLO defined
Contribution	Absent	Stated delta against the two base papers
Demo	Absent	90-second review demo scripted

1. Project idea

Sentinel watches security events in an isolated simulated network, connects related events into an attack path, predicts the attacker's likely next step, and produces a risk score so an analyst knows what needs attention first.

In one line: Event → Correlate → Update attack state → Predict next step → Score risk → Dashboard.

What it solves

- Shows how several small attack steps form one larger attack path.
- Connects events from different hosts instead of treating every alert separately.
- Uses public vulnerability and attack knowledge to add meaning to raw events.
- Predicts a probable next step instead of only reporting what already happened.
- Ranks risk so critical paths and assets get attention first.

2. Scope — the intrusion chain we are building for

NEW. *This section did not exist in Revision 1 and is the decision that unblocks everything else.*

Sentinel does not claim to detect all attacks. Attempting that is both indefensible at review and impossible to evaluate. Sentinel targets one named, multi-stage intrusion chain, expressed in MITRE ATT&CK terms:

Stage	ATT&CK ID	What happens in our lab
Initial Access	T1190 — Exploit Public-Facing Application	Attacker exploits Log4Shell (CVE-2021-44228) on the web application
Privilege Escalation	T1068 — Exploitation for Privilege Escalation	Attacker escalates from service account to root on the web host
Lateral Movement	T1021.004 — Remote Services: SSH	Attacker uses harvested credentials to reach the database host
Exfiltration	T1041 — Exfiltration Over C2 Channel	Attacker copies data out of the database host

Why anchor on Log4Shell specifically

- Trivially reproducible in a lab container, and exhaustively documented.
- Maps cleanly to T1190, so the ATT&CK linkage is real rather than asserted.
- It gives a genuine **CVE-2021-44228 → CWE-502 → CAPEC → ATT&CK** chain, which makes the knowledge graph load-bearing instead of decorative.

Say this at review, not "we detect attacks"

"Sentinel predicts the next stage of a multi-stage intrusion, scoped to a four-technique ATT&CK chain anchored on a real CVE. We evaluate on held-out scenario variants the model has not seen."

Scoping down is a strength at a zeroth review, not a weakness. An unscoped claim invites the question we cannot answer.

3. Terminology — the three graphs

CHANGED. Revision 1 used "attack graph" for two different objects, which is why the phase ordering looked wrong to the team.

Object	What it is	When it is built
Knowledge graph	Public cybersecurity concepts and their relationships: CVE, CWE, CAPEC, ATT&CK, with vulnerability description text attached	Build-time. Once, refreshed occasionally
Environment graph	Our lab: hosts, services, vulnerabilities, reachability between hosts	Build-time. Rebuilt when the lab changes
Attack state	The path observed so far in one live incident — which stages have happened, on which hosts, in what order	Runtime. One per incident, grows as events arrive

The distinction matters because the environment graph's vulnerability nodes are meaningless until they are linked to the knowledge graph. That is why the knowledge graph is now built first (Phase 4 before Phase 5, inverted from Revision 1).

Other terms

Term	Meaning
CVE (Common Vulnerabilities and Exposures)	A standard ID for one specific publicly known vulnerability
CWE (Common Weakness Enumeration)	The category or type of weakness a CVE belongs to
CAPEC (Common Attack Pattern Enumeration and Classification)	A catalogue of the attack patterns that abuse those weaknesses
MITRE ATT&CK	A knowledge base of real-world attacker tactics and techniques
Security-relevant event	An event type our scope config declares as evidence of a technique — see Phase 1
Transition probability	Empirical likelihood that one stage follows another, learned from our dataset
Risk score	Predicted likelihood combined with asset criticality and impact

4. The three data flows

CHANGED. *Revision 1 showed a single arrow that merged training and prediction. There are three distinct flows and confusing them makes the architecture impossible to reason about.*

A. Build-time (static, runs occasionally)

Public CVE / CWE / CAPEC / ATT&CK feeds → Knowledge graph

Lab network configuration + scanner output → Environment graph

Neither is rebuilt per event. Together they are the static context the model reasons against.

B. Offline training (runs during development, not in the demo)

Lab → Controlled experiments → Raw logs → Normalize

→ Label from experiment metadata → Sequence dataset

→ Train / validate / test → Saved model artifact

This flow produces one file: a trained model. It never runs during a live demo.

C. Runtime inference (what the demo actually shows)

Live event → Normalize → Filter to security-relevant

→ Correlate into an incident → Update attack state

→ Featurize against knowledge + environment graphs

→ Model inference → Probability + risk → Dashboard

Where the model sits

Centralized, and we should say so deliberately. Agents on each host do collection only. They ship to a central queue. A FastAPI service holds the graph state and the loaded model, and serves the dashboard.

This is what a SIEM is. The tradeoffs — single point of failure, no horizontal scaling — are real and we accept them as prototype constraints rather than pretending they are not there.

5. Phase 0 — Lock scope and write the demo first

NEW.

Before any lab is built, two artefacts get committed to the repository. Working backwards from the demo disciplines every later decision.

1. `scope.yaml` — the four techniques in Section 2, the event types that count as evidence for each, and the hosts in play. Versioned, reviewed, and referenced by every later phase.
2. `demo.md` — the 90-second review walkthrough from Section 16. If a proposed feature does not appear in the demo, it is out of scope for Phase 1–8.

6. Phase 1 — Build the lab

CHANGED. *Containers instead of VMs, Linux only instead of mixed Windows/Linux.*

Containers, not VMs

Revision 1 specified VirtualBox VMs. Containers on a dedicated Docker network cut the setup effort by more than half and reset in seconds rather than minutes, which matters when Phase 2 requires hundreds of reset-and-run cycles. VMs are only justified if we need true kernel-level isolation per host, which we do not for this scope.

Fallback if container telemetry proves insufficient: **three VMs** (attacker, web+app, database) rather than the five in Revision 1.

Linux only

Revision 1 included a Windows endpoint with Sysmon. Dropping it is not only about realism — production applications do run on Linux — it is about scope. One operating system means one telemetry schema to configure, normalize and label, instead of two logging stacks feeding two different event vocabularies into the same model.

Hosts

Host	Role	Exposes
attacker	Runs the adversary emulation tooling	—
web	Vulnerable web application (Log4Shell)	HTTP/HTTPS
db	Database server, holds the target data	Database wire protocol, SSH
collector	Log collection, queue, API, dashboard	HTTP

Telemetry

- Use **Falco** (eBPF-based, works correctly inside containers) or **auditd** if we fall back to VMs. Pick one and standardise.
- Ship everything to the collector. Filtering happens at the collector, not at the agent, so the raw capture stays available for re-labelling.
- Snapshot or image every host before experiments so resets are reliable and identical.

Defining "security-relevant events"

This was asked twice in team discussion and never answered. The answer is that **we define it ourselves, up front, as a versioned config file** — not something we discover from vendor metadata or system tags. Scope determines the filter.

Event type	Evidence for
Process creation (execve) with command line	T1190, T1068
Outbound network connection with destination	T1190, T1041
sshd authentication success / failure	T1021.004
setuid / sudo invocation	T1068
Write or read on declared sensitive paths	T1041

That table lives in scope.yaml. When scope changes, the filter changes with it, in one place.

7. Phase 2 — Controlled experiments

CHANGED, and this is the most important fix in the revision.

The circularity problem in Revision 1

Revision 1 scripted the sequence Initial Access → Privilege Escalation → Lateral Movement, then derived transition probabilities from those same runs. If we script the chain, the model "discovering" that the chain has 80% probability is just recovering our own script. We would be measuring whether the model can read our experiment config.

A reviewer will ask: what happens if the attacker does something else? Revision 1 had no answer. Branching gives us one.

The scenario library must branch

For a prediction problem to exist at all, the next stage must be genuinely uncertain given the current one. That requires the dataset to contain paths that diverge:

Scenario class	Path	Purpose
Full chain	Access → Escalate → Lateral → Exfiltrate	The positive case
Early stop	Access, then nothing further	Not every intrusion continues
Local objective	Access → Escalate → Exfiltrate from same host	Escalation does not always imply lateral movement
Failed escalation	Access → Escalate (fails) → retry → stop	Attacks fail; the model must tolerate it
Benign lookalike	Legitimate admin SSH between hosts	Lateral movement is not the only reason for SSH
Background only	Normal application and user traffic	Negative class

Without the last three rows in particular, the model has no way to learn when NOT to predict the next stage, and precision on the benign class will be meaningless.

How the attacks actually run

Revision 1 never specified this, and it matters more than it looks. Use **MITRE Caldera** for automated multi-step adversary emulation, or **Atomic Red Team** for per-technique scripted atomics.

Why this is the highest-leverage decision in the plan

Both tools emit an operation log recording which technique ran, at what time, on which host. That log is ground truth.

It hands us Phase 3's labelling problem already solved — no manual annotation of which log lines belong to which stage. Choosing the emulation tool well removes the single most tedious task in the project.

Experiment procedure

- Automate the cycle: choose scenario → reset lab → start background activity → run emulation → collect logs → archive with experiment ID → reset.
- Vary target host, timing, sequence and background load so the dataset is not one pattern repeated.

- Record scenario class, start time, hosts involved, and the emulation tool's operation log alongside the raw telemetry.
- Aim for enough runs per scenario class that the rarest class still has a usable test set — plan the count backwards from that, not forwards from convenience.

8. Phase 3 — Build the ML dataset

CHANGED. *The splitting strategy in Revision 1 would have leaked and produced meaningless accuracy.*

Processing

- Keep each experiment archived separately and immutably.
- Normalize timestamp, source host, destination host, user, process and event type into one schema.
- Label stages from the emulation tool's operation log, not by hand.
- Preserve event order and timestamps so sequences can be reconstructed.
- Build sequence examples: the events so far, the current state, and the next stage where one exists.

Do not randomly split event rows

Revision 1 said "split the dataset into training, validation and test sets" immediately after describing a table of per-event rows. Splitting those rows randomly puts events from the same attack run on both sides of the split. The model sees most of an incident in training and is then asked about the rest of the same incident at test time. That reports near-perfect accuracy which means nothing.

Split at the EXPERIMENT level. Better still, hold out entire scenario variants, so the test set measures generalization to chains the model has never seen rather than memorization of ones it has.

Over-cleaning

A caution raised in team discussion and worth keeping: refining the dataset too aggressively stops it mirroring reality. Remove malformed records and true duplicates, but keep the noise, the background traffic and the failed attempts. A model trained only on clean, unambiguous sequences will not survive contact with a live event stream.

9. Phase 4 — Build the knowledge graph

CHANGED — moved ahead of the environment graph.

- Pull structured public data: NVD/CVE, CWE, CAPEC and MITRE ATT&CK. Download and parse; never hand-type entries.
- Create nodes for CVEs, CWEs, CAPEC patterns and ATT&CK techniques.
- Create edges using the relationships the sources already provide.
- Keep vulnerability description **text** attached to the vulnerability nodes — this is what the text-enhanced model in Phase 6 consumes, and without it there is no text-enhancement to speak of.
- Restrict the initial import to the neighbourhood of our four techniques. Importing all of NVD produces a graph we cannot inspect or debug.

Tool: NetworkX for the prototype. Move to Neo4j only if traversal performance becomes a measured problem, not preemptively.

10. Phase 5 — Build the environment graph

Nodes are hosts, services and the vulnerabilities present on them. Edges are relationships: runs, has-vulnerability, can-reach.

Each vulnerability node links into the knowledge graph built in Phase 4 — that link is what turns "the web host has CVE-2021-44228" into "the web host is exploitable via T1190, which commonly precedes T1068."

Sources: the lab configuration we wrote ourselves, plus scanner output against the lab.

11. Phase 6 — Graph ML and next-step prediction

CHANGED. *Baselines added; they were absent from Revision 1.*

The model receives the current attack state together with its graph context, and ranks the possible next techniques by probability.

- Featurize graph structure plus the vulnerability description text into node representations.
- A Graph Attention Network is the candidate model, following Gao et al. Train on the training split, tune on validation, evaluate once on held-out test.
- The model estimates likely behaviour from learned patterns. It does not know the future and we should never present it as certainty.

Build the baselines before the GAT

Two baselines are mandatory, and both are a day's work:

1. Majority class — always predict the most common next stage.
2. Markov chain over technique transitions — no graph, no neural network, just counted transition frequencies.

If the GAT does not clearly beat the Markov chain, the graph and the neural network are not earning their place, and the honest move is to report that. We need to know this in December, not in May. Discovering it late is the single largest schedule risk in the project.

12. Phase 7 — Probabilistic correlation and risk

Correlation

Deciding whether a new event belongs to an existing incident or starts a new one. The decision uses event timing, source and destination hosts, event type, reachability in the environment graph, and the current attack state — following the approach in Ben Fredj and Cheikhrouhou.

Probabilities

Transition probabilities are estimated from the controlled sequences. These are empirical probabilities from our dataset, and we must label them that way in the dashboard and the report. They are not probabilities of real-world attacker behaviour, and claiming otherwise is the kind of overreach a panel will catch.

Risk

Risk combines predicted likelihood, vulnerability severity, asset criticality and path impact. Choose the formula early, write it down, and keep it simple enough to explain in one sentence at review. A weighted product of likelihood and asset criticality is defensible; an unexplainable composite is not.

13. Phase 8 — Runtime pipeline and dashboard

Per-event path

3. Event arrives at the collector.
4. Normalize to the common schema.
5. Filter against the security-relevant event config.
6. Correlate into an existing incident, or open a new one.
7. Update that incident's attack state.
8. If the state changed materially, run inference.
9. Compute risk and push the update to the dashboard.

Do not run the GNN on every event

This was the unanswered question about log volume and latency, and the architecture answers it.

Steps 1–5 are cheap and run per event — milliseconds each. Step 6 fires only when the attack state changes materially: a new host involved, a new technique observed, a new edge in the state. That is one inference every few seconds during an active incident, not tens of thousands per second.

Separating the cheap per-event path from the expensive state-change path is what makes "near real time" achievable rather than aspirational.

Latency budget

Set the target explicitly and then report what we actually measure:

- p95 event-to-dashboard under **5 seconds** at a stated sustained event rate.
- Report the sustained rate we actually achieve on lab hardware. Do not claim enterprise scale; state our measured number and note that scaling is future work.

Dashboard

- Current attack path as a graph, with completed stages distinguished from predicted ones.
- Predicted next technique with its probability, explicitly labelled as empirical.
- Risk score and ranked assets at risk.
- Suggested remediation for the predicted next step.

14. Evaluation plan

NEW. Revision 1 contained no metrics anywhere, which meant there was no way to tell whether the project had succeeded.

Component	Metric	Compared against
Next-step prediction	Top-1 and Top-3 accuracy; macro-F1 (classes are imbalanced, so plain accuracy will mislead)	Majority class, Markov chain
Event correlation	Precision and recall of assigning events to the correct incident	Time-window-only correlation
Risk ranking	Whether truly critical paths rank above non-critical ones	Raw CVSS ordering

Component	Metric	Compared against
Pipeline	p50 / p95 / p99 event-to-dashboard latency; sustained event rate	Stated SLO of 5s at p95

Every number reported comes from the held-out test set, evaluated once. Tuning against the test set and then reporting it is the most common way student projects overstate results, and it is easy to do accidentally.

15. What we contribute over the base papers

NEW. *This was raised in team discussion — how our work builds on current research — and Revision 1 did not address it.*

Paper	What it does	What it does not do
Gao et al. (2026), Electronics 15(3), 552	Text-enhanced graph attention over a cybersecurity knowledge graph; recovers attack paths structural methods miss	Static knowledge graph, evaluated offline on graph triples — not driven by live observed events
Ben Fredj & Cheikhrouhou (2026), J. Supercomputing 82, 262	Live probabilistic alert correlation over attack graphs, polynomial-time, real-time capable	No text semantics; correlation over alerts rather than a text-enriched knowledge representation

Our claim, stated narrowly

Sentinel sits at the intersection: it uses the text-enriched knowledge-graph representation from the first paper to inform RUNTIME next-step prediction over observed events, in the near-real-time correlation setting of the second.

This is a modest and honest claim, which is exactly what a zeroth-review panel wants to hear. Do not inflate it into a claim of novelty over the whole field.

16. The review demo

NEW. *Decide now what the reviewer physically watches. Every other decision should serve this.*

Time	On screen	What we say
0:00	Dashboard, quiet. Four hosts on the environment graph	"Isolated lab. Nothing happening yet."
0:15	Attacker container fires the Log4Shell exploit	"We are exploiting a real CVE on the web application."
0:25	Initial Access appears on the path; prediction shows Privilege Escalation at $p \approx 0.7$	"Sentinel has correlated the events and predicts the next stage."
0:45	Attacker escalates. Path advances; prediction updates to Lateral Movement, DB host flagged	"The prediction updated as the incident progressed."
1:10	Risk panel ranks the database host highest	"Risk combines likelihood with asset criticality."
1:30	Remediation suggestion for the predicted step	"And this is what an analyst would act on."

A second run of the benign-lookalike scenario, where Sentinel correctly does not raise an incident on legitimate admin SSH, is worth 30 seconds if time allows. It pre-empt the obvious question about false positives.

17. Execution order

```

PHASE 0  — Lock scope.yaml and demo.md
      ↓
PHASE 1  — Containerised Linux lab + Falco telemetry
      ↓
PHASE 2  — Branching experiments via Caldera / Atomic Red Team
      ↓
PHASE 3  — Label from operation logs; split by experiment
      ↓
PHASE 4  — Knowledge graph (CVE / CWE / CAPEC / ATT&CK + text)
      ↓
PHASE 5  — Environment graph, linked into the knowledge graph
      ↓
PHASE 6  — Baselines first, then the GAT
      ↓
PHASE 7  — Correlation, probabilities, risk scoring
      ↓
PHASE 8  — Runtime pipeline + dashboard

```

Phases 4 and 5 can proceed in parallel with Phases 1–3, since the knowledge graph depends on public data rather than on our lab. That parallelism is worth exploiting given the schedule.

18. Open decisions

These are genuinely undecided. Each should be closed by a named owner rather than left to the next group call.

Decision	Options	Blocks
Containers or VMs	Docker + Falco / three VMs + auditd	Phase 1
Emulation tool	Caldera / Atomic Red Team	Phase 2
Graph store	NetworkX / Neo4j	Phase 4
Risk formula	Weighted product / learned weighting	Phase 7
Runs per scenario class	Driven by the size of the rarest test class	Phase 2

On process

Only one of these — the scope decision in Section 2 — actually blocks everything else. It is already settled in this document; it needs confirmation, not rediscovery.

The rest can be decided asynchronously by their owners. Waiting for all four members to be simultaneously available has already cost several days. Put decisions in a shared document with a comment deadline and reserve calls for genuine disagreement.

19. Safety

All attack simulation stays inside the isolated lab network, against systems created for this project and authorised by the department. Nothing is run against college infrastructure, personal devices or any external system.

The lab network has no route to the internet or the campus network beyond what is explicitly required to pull public vulnerability data, and that pull happens from the collector host only.

The objective is controlled data generation for defensive analysis. Exploit code stays in the repository's lab directory and is not distributed.